Sending the executing process to the background

Here is a simple tip that will help you to send the process running in the foreground to the background. Follow the steps shown below:

1. Suspend the process running in the foreground using 'Ctrl + z'
2. Move the process to the background using 'bg'
3. Re-parent the process to init using 'disown %<job_id>'. *Job_id* can be listed using the command 'jobs'

Now the process is under *init* when the terminal is exited. This can be checked with 'ps -ef | grep -i <process name>' from the new terminal.

– Parthiban Nallathambi,
parthitce@gmail.com

Executing a command without history

We all know that any command we enter at the terminal prompt gets logged into a file called *~/.bash_history*. To skip this logging step while running a command, just prefix a space, and the command will not be logged:

```
[space] {command}

Example:[space] ls
```

And afterwards, run the *{history}* command to check. You won't find any commands that have been executed with a leading space.

– Hussain Kathawala,
Matrix7886@hotmail.com

How to truncate the file's size to zero

Many a time, it is necessary to truncate an unwanted file's contents, like in the case of old log files. One of the easiest ways to do this is to open a file in the text editor and delete all the lines. Though this solution seems easy, it is not practical. What if the log file size is 50MB or more? In most cases, your text editor will become unresponsive.

But with the GNU/Linux command line, we can do this task very easily and efficiently. Shown below is a simple example that illustrates this.

Let us first create a text file with some junk data, as follows:

```
[bash]$ echo "Junk data" > junk.txt
[bash]$ cat junk.txt
Junk data
```

Now truncate the data from the file using shell redirection:

```
[bash]$ :> junk.txt
[bash]$ cat junk.txt
```

The output shows that this is an empty file.

In the above example, *colon(:)* is the null statement. With modern shells, we can skip this, but it is required by some older shells.

– Narendra Kangralkar,
narendrakangralkar@gmail.com

Timing out processes

It often happens that we want a certain command to execute for only a limited period of time. Let's suppose I want to listen to a music player for only half an hour, or that I want to find something for not more than 15 seconds.

In bash scripts, timeout becomes very handy while experimenting with scripts that can take too much time to complete.

The syntax for this is:

```
timeout 10s {command}
```

The examples are:

```
timeout 30m mplayer
timeout 15s find
```

– Manish Jain,
manish.msiclub@gmail.com

END

Share Your Linux Recipes!

The joy of using Linux is in finding ways to get around problems—take them head on, defeat them! We invite you to share your tips and tricks with us for publication in *OSFY* so that they can reach a wider audience. Your tips could be related to administration, programming, troubleshooting or general tweaking. Submit them at *www.opensourceforu.com*. The sender of each published tip will get a T-shirt.